Manual for Artificial Intelligence and Applications Lab
BCA VI Sem

1. Write a program to implement breadth first search using python.

```
#Using a Python dictionary to act as an adjacency list
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}
visited = []  # List to keep track of visited nodes.
queue = []  # Initialize a queue

def bfs(visited, graph, node):
   visited.append(node)
   queue.append(node)

   while queue:
      s = queue.pop(0)
      print(s, end=" ")

      for neighbour in graph[s]:
         if neighbour not in visited:
            visited.append(neighbour)
            queue.append(neighbour)

# Driver Code
bfs(visited, graph, 'A')
```

2. Write a Program to Implement Depth First Search using Python.

```
#Using a Python dictionary to act as an adjacency list
graph = {
   'A' : ['B','C'],
   'B' : ['D', 'E'],
   'C' : ['F'],
   'D' : [],
   'E' : ['F'],
   'F' : []
}
visited = set() # Set to keep track of visited nodes.

def dfs(visited, graph, node):
   if node not in visited:
      print (node)
      visited.add(node)

      for neighbour in graph[node]:
         dfs(visited, graph, neighbour)
# Driver Code
```

```
dfs(visited, graph, 'A')
```

3. Write a Program to Implement 8-Puzzle problem using Python

```
class Solution:
    def solve(self, board):
        dict = {}
        flatten = []

        for i in range(len(board)):
            flatten += board[i]
        flatten = tuple(flatten)

        dict[flatten] = 0

        if flatten == (0, 1, 2, 3, 4, 5, 6, 7, 8):
            return 0

        return self.get_paths(dict)

    def get_paths(self, dict):
        cnt = 0
        while True:
            current_nodes = [x for x in dict if dict[x] == cnt]
            if len(current_nodes) == 0:
                return -1

            for node in current_nodes:
                next_moves = self.find_next(node)
                for move in next_moves:
                    if move not in dict:
                        dict[move] = cnt + 1
                        if move == (0, 1, 2, 3, 4, 5, 6, 7, 8):
                            return cnt + 1
            cnt += 1

    def find_next(self, node):
        moves = {0: [1, 3],
                 1: [0, 2, 4],
                 2: [1, 5],
                 3: [0, 4, 6],
                 4: [1, 3, 5, 7],
                 5: [2, 4, 8],
                 6: [3, 7],
                 7: [4, 6, 8],
                 8: [5, 7]}
        results = []
        pos_0 = node.index(0)
        for move in moves[pos_0]:
            new_node = list(node)
            new_node[move], new_node[pos_0] = new_node[pos_0], new_node[move]
            results.append(tuple(new_node))
        return results
```

```
ob = Solution()
matrix = [
   [3, 1, 2],
   [4, 7, 5],
   [6, 8, 0]
]

print(ob.solve(matrix))
```

4. Write a program to implement N-queens problem using python.

```
N = 4
def printSolution(board):
   for i in range(N):
      for j in range(N):
         print(board[i][j], end=' ')
      print()

def isSafe(board, row, col):
   # Check this row on left side
   for i in range(col):
      if board[row][i] == 1:
         return False
   # Check upper diagonal on left side
   for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
      if board[i][j] == 1:
         return False
   # Check lower diagonal on left side
   for i, j in zip(range(row, N, 1), range(col, -1, -1)):
      if board[i][j] == 1:
         return False
   return True

def solveNQUtil(board, col):
   # base case: If all queens are placed then return true
   if col >= N:
      return True
   for i in range(N):
      if isSafe(board, i, col):
         # Place this queen in board[i][col]
         board[i][col] = 1
         # recur to place rest of the queens
         if solveNQUtil(board, col + 1):
            return True
         # If placing queen in board[i][col] doesn't lead to a solution, then remove queen from
board[i][col]
         board[i][col] = 0
   # if the queen cannot be placed in any row in this column col then return false
   return False

def solveNQ():
   board = [[0, 0, 0, 0],
         [0, 0, 0, 0],
```

```
            [0, 0, 0, 0],
            [0, 0, 0, 0]]
    if not solveNQUtil(board, 0):
        print("Solution does not exist")
        return False
    printSolution(board)
    return True

# Driver program to test the above function
solveNQ()
```

5. Write a Program to Implement Alpha-Beta Pruning using Python.

```
# Initial values of Alpha and Beta
MAX, MIN = 1000, -1000

# Returns the optimal value for the current player
# (Initially called for root and maximizer)
def minimax(depth, nodeIndex, maximizingPlayer, values, alpha, beta):
    # Terminating condition. i.e., leaf node is reached
    if depth == 3:
        return values[nodeIndex]

    if maximizingPlayer:
        best = MIN
        # Recur for left and right children
        for i in range(0, 2):
            val = minimax(depth + 1, nodeIndex * 2 + i, False, values, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            # Alpha Beta Pruning
            if beta <= alpha:
                break
        return best
    else:
        best = MAX
        # Recur for left and right children
        for i in range(0, 2):
            val = minimax(depth + 1, nodeIndex * 2 + i, True, values, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            # Alpha Beta Pruning
            if beta <= alpha:
                break
        return best

# Driver Code
if __name__ == "__main__":
    values = [3, 5, 6, 9, 1, 2, 0, -1]
    print("The optimal value is :", minimax(0, 0, True, values, MIN, MAX))
```

6. Write a program to implement forward chaining algorithm.

```
database = ["Croaks", "Eat Flies", "Shrimps", "Sings"]
knowbase = ["Frog", "Canary", "Green", "Yellow"]
def display():
        print("\n X is \n1..Croaks \n2.Eat Flies \n3.shrimps \n4.Sings ", end=")
        print("\n Select One ", end=")
def main():
        print("*-----Forward--Chaining-----*", end=")
        display()
        x = int(input())
        print(" \n", end=")
        if x == 1 or x == 2:
                print(" Chance Of Frog ", end=")
        elif x == 3 or x == 4:
                print(" Chance of Canary ", end=")
        else:
                print("\n-------In Valid Option Select --------", end=")
        if x >= 1 and x <= 4:
                print("\n X is ", end=")
                print(database[x-1], end=")
                print("\n Color Is 1.Green 2.Yellow", end=")
                print("\n Select Option ", end=")
                k = int(input())
                if k == 1 and (x == 1 or x == 2): # frog0 and green1
                        print(" yes it is ", end=")
                        print(knowbase[0], end=")
                        print(" And Color Is ", end=")
                        print(knowbase[2], end=")
                elif k == 2 and (x == 3 or x == 4): # canary1 and yellow3
                        print(" yes it is ", end=")
                        print(knowbase[1], end=")
                        print(" And Color Is ", end=")
                        print(knowbase[3], end=")
                else:
                        print("\n---InValid Knowledge Database", end=")
if __name__ == "__main__":
main()
```

7. Write a program to implement backward chaining algorithm.

```
database = ["Croaks", "Eat Flies", "Shrimps", "Sings"]
knowbase = ["Frog", "Canary"]
color = ["Green", "Yellow"]
def display():
        print("\n X is \n1.frog \n2.canary ", end=")
        print("\n Select One ", end=")
def main():
```

```
        print("*-----Backward--Chaining-----*", end=")
        display()
x = int(input())
print(" \n", end=")
if x == 1:
        print(" Chance Of eating flies ", end=")
elif x == 2:
        print(" Chance of shrimping ", end=")
else:
        print("\n-------In Valid Option Select --------", end=")
        if x >= 1 and x <= 2:
                print("\n X is ", end=")
                print(knowbase[x-1], end=")
                print("\n1.green \n2.yellow")
                k = int(input())
        if k == 1 and x == 1: # frog0 and green1
                print(" yes it is in ", end=")
                print(color[0], end=")
                print(" colour and will ", end=")
                print(database[0])
        elif k == 2 and x == 2: # canary1 and yellow3
                print(" yes it is in", end=")
                print(color[1], end=")
                print(" Colour and will ", end=")
                print(database[1])
        else:
                print("\n---InValid Knowledge Database", end=")
if __name__ == "__main__":
main()
```

8. Write a program to implement KNN algorithm to classify Iris dataset. Print both correct and wrong predictions.

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Replace 'your_file_path.csv' with the actual path to your Iris dataset file
file_path = 'your_file_path.csv'

# Load the Iris dataset from the specified location
iris_data = pd.read_csv(file_path)

# Separate features (X) and target variable (y)
X = iris_data.iloc[:, :-1].values  # Features
y = iris_data.iloc[:, -1].values   # Target variable
```

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the k-NN classifier
knn = KNeighborsClassifier(n_neighbors=3)  # You can adjust the number of neighbors as needed

# Train the classifier
knn.fit(X_train, y_train)

# Predict on the test set
y_pred = knn.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

9. Train a random data sample using linear regression model and plot the graph.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Generate some sample data
np.random.seed(0)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

# Train the linear regression model
model = LinearRegression()
model.fit(X, y)

# Make predictions
X_new = np.array([[0], [2]])
y_pred = model.predict(X_new)

# Plot the data points and the linear regression line
plt.scatter(X, y, color='blue')
plt.plot(X_new, y_pred, color='red')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Linear Regression')
plt.show()
```

10. Implement the naïve Bayesian classifier for a sample training data set stored as a .csv file. Compute the accuracy of the classifier, considering few test data sets.

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

# Replace 'your_file_path.csv' with the actual path to your Iris dataset file
file_path = 'your_file_path.csv'

# Load the Iris dataset from the specified location
iris_data = pd.read_csv(file_path)

# Separate features (X) and target variable (y)
X = iris_data.iloc[:, :-1].values  # Features
y = iris_data.iloc[:, -1].values   # Target variable

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features (not necessary for Naive Bayes, but doesn't hurt)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the Naive Bayes classifier
naive_bayes = GaussianNB()

# Train the classifier
naive_bayes.fit(X_train, y_train)

# Predict on the test set
y_pred = naive_bayes.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

11. Demonstrate the working of SVM classifier for a suitable data set(e.g., iris dataset)

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.datasets import load_iris

# Replace 'your_file_path.csv' with the actual path to your Iris dataset file
file_path = 'your_file_path.csv'

# Load the Iris dataset from the specified location
iris_data = pd.read_csv(file_path)

# Separate features (X) and target variable (y)
X = iris_data.iloc[:, :-1].values  # Features
y = iris_data.iloc[:, -1].values   # Target variable

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features (SVM is sensitive to feature scaling)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the SVM classifier
svm_classifier = SVC(kernel='linear', random_state=42)  # Using a linear kernel

# Train the classifier
svm_classifier.fit(X_train, y_train)

# Predict on the test set
y_pred = svm_classifier.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Display confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

12. Build a sample binary image classification model (cat and dog).

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# Now, from the downloaded dataset, create train, validation, and test folders, and split cat and
#dog images into these folders in the ratio of 70:20:10.

# Define directories for training and validation data
train_dir = 'path_to_training_data_directory'
validation_dir = 'path_to_validation_data_directory'

# Define constants
batch_size = 32
img_height = 150
img_width = 150
epochs = 10

# Create image data generators
train_datagen = ImageDataGenerator(rescale=1./255)
validation_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)

validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(img_height, img_width),
    batch_size=batch_size,
    class_mode='binary'
)

# Build a simple neural network model
model = Sequential([
    Flatten(input_shape=(img_height, img_width, 3)),
    Dense(128, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Train the model
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // batch_size
)
```

NOTE:
Cat Dog Dataset for Program No. 12 can be downloaded from the following link
https://www.kaggle.com/datasets/tongpython/cat-and-dog

Credits:
Dr. Honnaraju
Associate Professor
Dept. of AI and DS
MIT Mysore

Prof. Shilpa P
Assistant Professor
Govt. First Grade College
Kuvempunagar